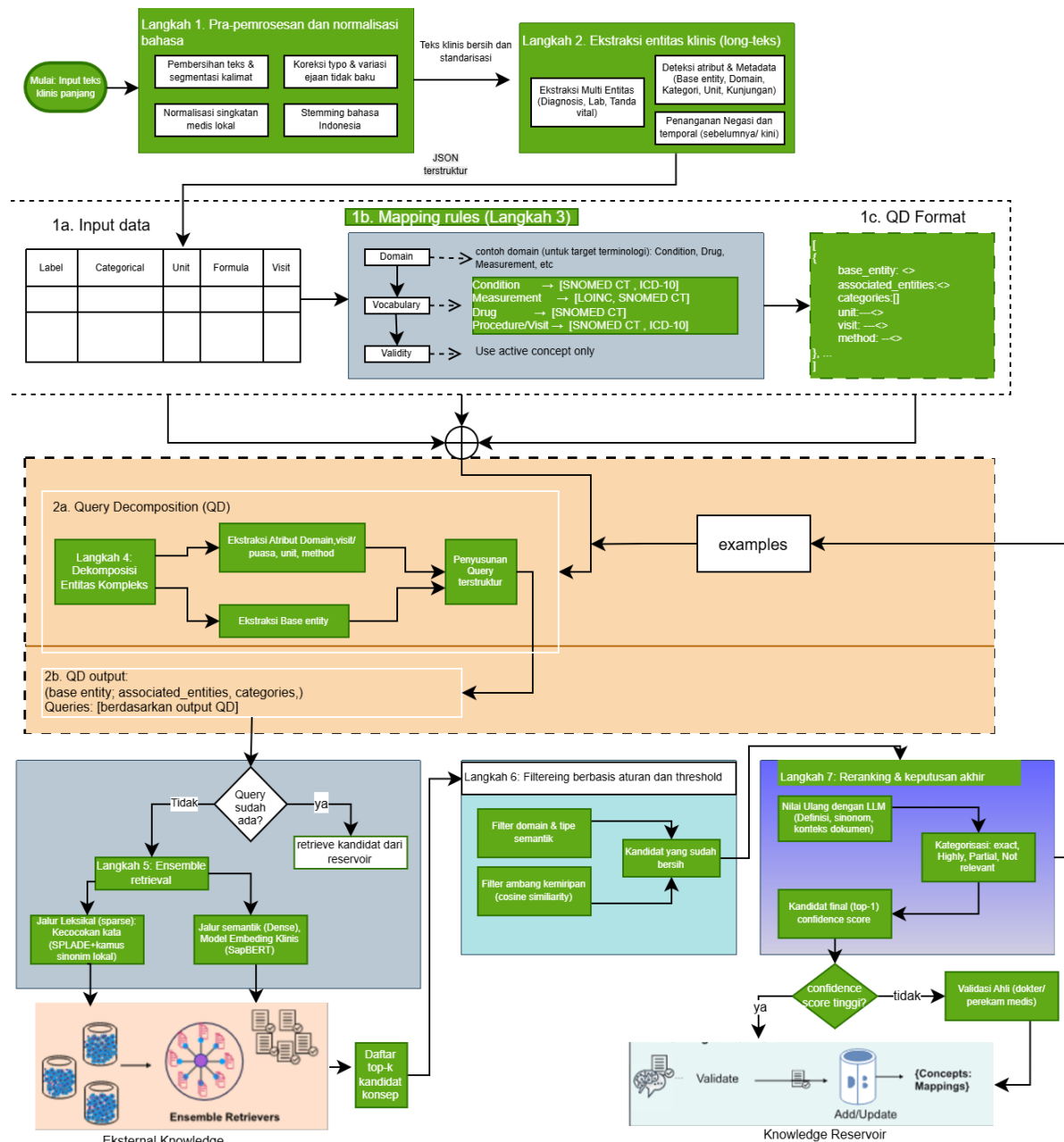
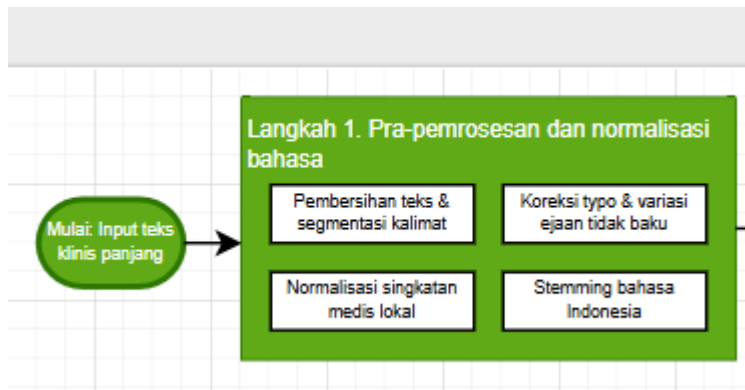
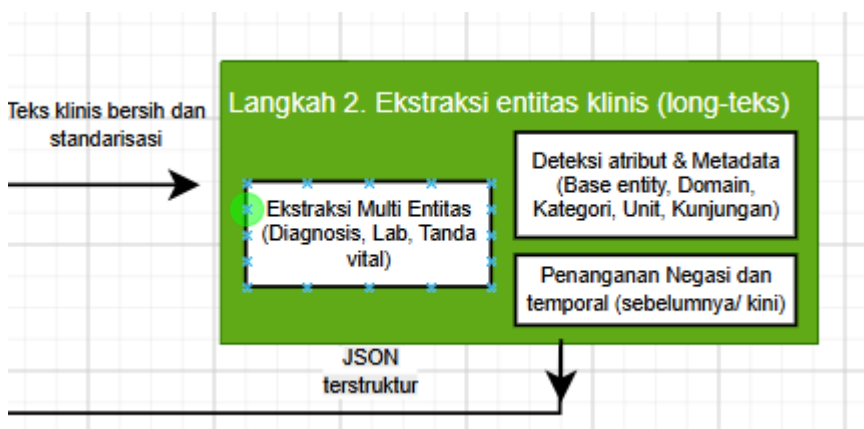






1. Pembersihan Teks → text-normalization (GitHub). Modul Python yang menyediakan fungsi pembersihan untuk teks media sosial, seperti menghapus emotikon, karakter berulang, dan baris baru → <https://github.com/yasirutomo/text-normalization>
2. Segmentasi Kalimat → Transformer-based Text Segmentation. Skrip segmentasi kalimat berbasis aturan yang khusus dirancang untuk bahasa Indonesia (<https://doi.org/10.1145/3789418.3789430>) → [https://github.com/blacfli/indo\\_text\\_segmentation](https://github.com/blacfli/indo_text_segmentation)
3. Koreksi Typo & Ejaan Tidak Baku → indo-normalizer. Pustaka Python yang sangat baik untuk menormalisasi teks informal Indonesia, seperti mengubah 'bgt' → 'banget', 'gpp' → 'gapapa', dan memperbaiki pengulangan huruf ('akuuuu' → 'aku') → <https://github.com/drestanto/indo-normalizer>
4. Normalisasi Singkatan → Kamus Singkatan Medis. Membuat kamus khusus yang berisi pasangan singkatan medis umum (misal: 'TD' → 'tekanan darah', 'HB' → 'hemoglobin', 'DM' → 'diabetes melitus'). → kamus singkatan medis diambil dari <http://61.8.75.226/itblog/attachments/article/1387/lampiran%20Daftar%20Singkatan%20dan%20Simbol%20Rekam%20Medis.pdf>
5. Stemming Bahasa Indonesia → Sastrawi. Pustaka Python standar untuk stemming bahasa Indonesia. Mengimplementasikan algoritma Nazief & Adriani yang mampu mengembalikan kata berimbuhan menjadi kata dasar (misal: 'meminum' → 'minum') → <https://github.com/sastrawi/sastrawi>



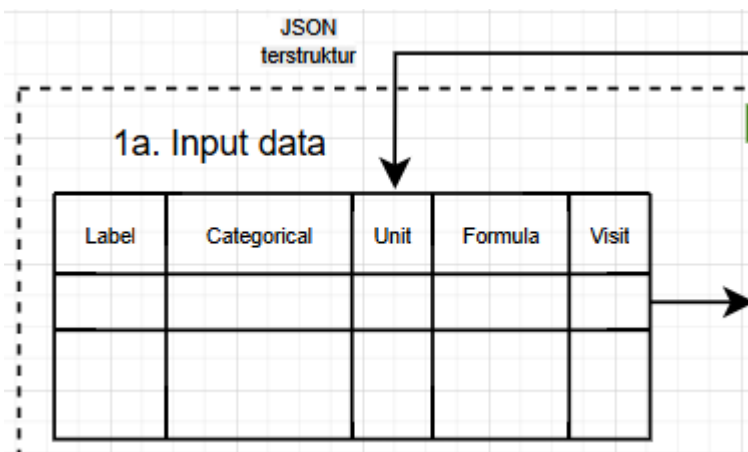
Langkah 2 menggunakan LLM atau Model NER khusus untuk mendeteksi atribut dan metadata berupa base entity, domain, kategori, unit, dan kunjungan. Target atribut adalah berupa multi entitas dan dapat menangani negasi dan konteks temporal.

[hapus kotak input LLM/Model NER khusus]

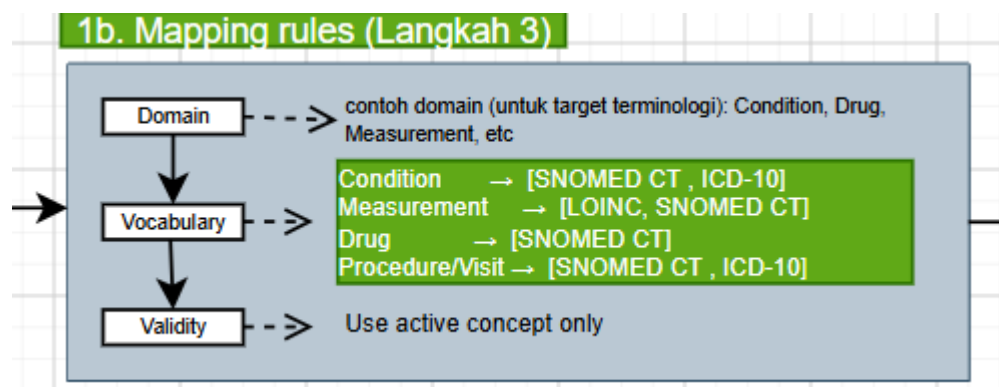
Kandidat tools:

1. Ekstraksi Multi-Entitas (Diagnosis, Lab, Tanda Vital) → Medical LLMs. Menggunakan model LLM yang sudah dilatih pada korpus medis, seperti Numind Medical NER LLM, yang dapat langsung mengekstrak entitas klinis dalam format JSON [https://nlp.johnsnowlabs.com/2026/01/07/jsl\\_meds\\_ner\\_2b\\_q8\\_v1\\_en.html](https://nlp.johnsnowlabs.com/2026/01/07/jsl_meds_ner_2b_q8_v1_en.html) → **berlisensi** (atau bisa menggunakan lib LangExtract)
2. Deteksi Atribut & Metadata (base entity, domain, kategori, unit, kunjungan) → Few-shot Learning. Memberikan beberapa contoh pasangan input-output dalam prompt untuk menunjukkan format JSON yang Anda inginkan di LLM → API (Togetherai)
3. **Penanganan Konteks Negasi** (misal: 'tidak ada') → Spark NLP's Contextual Assertion. Komponen dari pustaka Spark NLP yang secara khusus dirancang untuk menganalisis konteks klinis, termasuk negasi, keraguan, dan temporalitas → <https://www.johnsnowlabs.com/using-contextual-assertion-for-clinical-text-analysis-a-comprehensive-guide/>
4. **Penanganan Konteks Negasi** (misal: 'tidak ada') → Prompt Spesifik Dalam instruksi LLM, tambahkan: 'Perhatikan kata-kata seperti tidak, bukan, tanpa sebagai penanda negasi. Jika entitas disebut dalam konteks negasi, beri label assertion: negated.' → API (Togetherai) **jadi pakai 4 atau 5?**
5. **Penanganan Konteks Temporal** (sebelumnya / kini) → Prompt Spesifik. Tambahkan dalam prompt: 'Perhatikan kata-kata seperti sebelumnya, dulu, kini, saat ini sebagai penanda waktu. Beri label temporal: past untuk masa lalu, present untuk saat ini.' → API (Togetherai)
6. **Penanganan Konteks Temporal** (sebelumnya / kini) → Custom Rule-based. Buat aturan sederhana dengan regex untuk mendeteksi kata-kata temporal dan menandai entitas di sekitarnya → **jadi pakai 6 atau 7?**
7. Output JSON Terstruktur → JSON Mode pada LLM. Beberapa model (seperti GPT-4, Claude, atau Gemini) mendukung mode JSON dengan skema output dan model akan mengembalikan JSON yang sesuai → (atau bisa menggunakan lib LangExtract)

Selanjutnya masuk kedalam algoritma CDE Mapper Gilani:



Setelah teks klinis Panjang dalam Bahasa Indonesia di-preproses (langkah 1 dan 2) akan dihasilkan output berupa data terstruktur dalam bentuk JSON dengan kolom label, categorical, unit, Formula dan Visit. Data ini akan menjadi input untuk proses berikutnya. **Jelaskan apa itu label, categorical, unit, Formula dan Visit.**



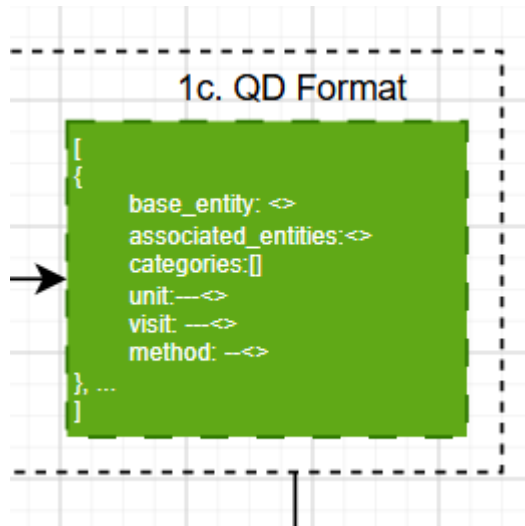
Data JSON terstruktur kemudian dikenakan proses Mapping rules. Langkah ini di tandai dengan proses 1b pada arsitektur asli Gilani, yaitu memeriksa Domain entity (apakah termasuk condition, Drugs, Measurement, atau domain yg lain)

Berdasarkan domain yang dikenali, selanjutnya dilakukan penentuan target mapping kepada vocabulary yang sesuai. Pada arsitektur Gilani (Langkah 1b), mapping dilakukan ke **vocabulary** SNOMED, LOINC, ICD, ATC, MedDRA, RxNorm/RxNorm-Extension, MeSH, UCUM, OMOP Extension, and OMOP Genomic. Sedangkan pada arsitektur adaptasi (Langkah 3) mapping hanya dilakukan ke 3 vocabulary utama yaitu SNOMEDCT, LOINC dan ICD-10.

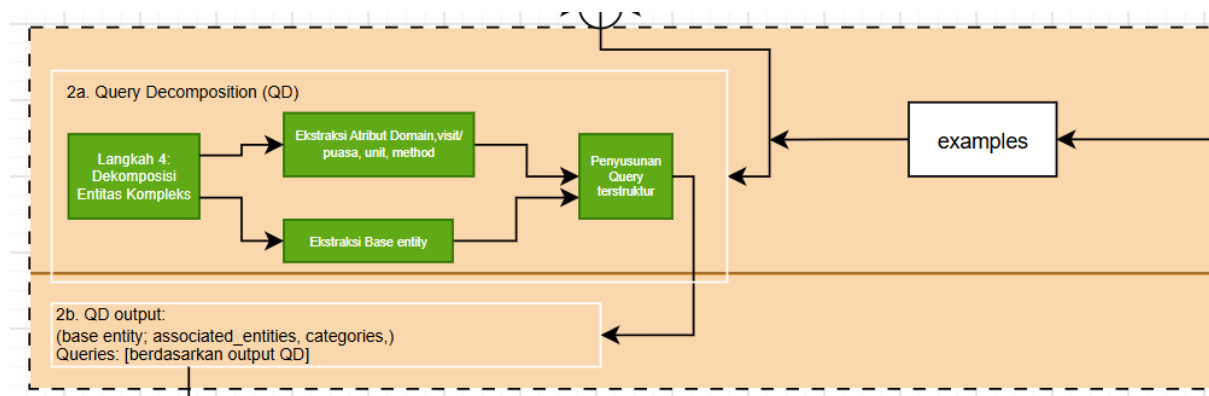
Domain → Terminologi Target (Prioritas)  
 Condition → [SNOMED CT (Utama), ICD-10 (Pelaporan)]  
 Measurement → [LOINC (Utama), SNOMED CT (Alternatif)]  
 Drug → [RxNorm / ATC (Utama), SNOMED CT]

Procedure/Visit → [SNOMED CT (Utama), ICD-10]

Validitas mapping dikerjakan menggunakan penggunaan konsep aktif saja (sama dengan yang dikerjakan oleh Gilani). Langkah ini memastikan bahwa framework akan menggunakan konsep yg digunakan secara aktif dalam terminology target sehingga mapping akan menjadi valid.



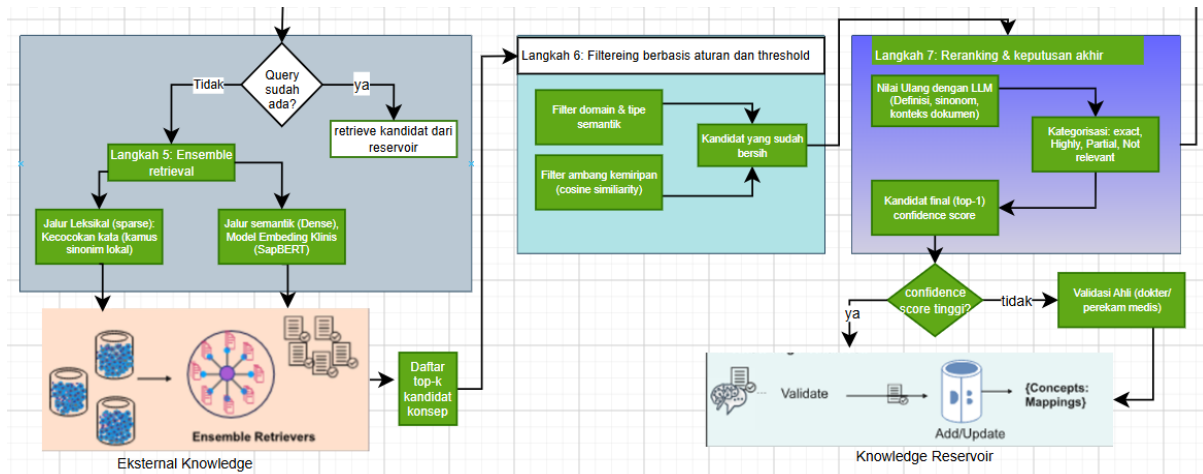
Selanjutnya, pada Langkah 1.c dilakukan transformasi format JSON untuk siap masuk ke tahap berikutnya yaitu Query decomposition. Perbedaan dengan framework asli dari Gilani, pada framework usulan akan didapatkan beberapa set entitas sehingga output dari 1.c tidak hanya 1 set entitas saja.



Pada Langkah 2a Gilani, LLM membaca metadata (label, categories, unit, visit, formula) sedangkan pada framework usulan, dilakukan pemecahan entitas kompleks menjadi komponen-komponen logis. **Tujuan utamanya adalah mengubah *query mentah* yang bisa jadi ambigu atau terlalu spesifik (misal: "caisson disease" + "main cause" + "status 0/1") menjadi *unit-unit pencarian (sub-queries)* yang atomik dan jelas.**

Langkah 2a atau Langkah 4 pada framework usulan akan menghasilkan dua hal sekaligus: yaitu **Struktur JSON Terdekomposisi** dan **Daftar Sub-Queries (Pertanyaan Pencarian)**.

Gilani et al. (2025) menerapkan Query Decomposition pada 1 baris data dictionary terstruktur (1 entitas per proses). Penelitian ini mengadaptasi mekanisme tersebut untuk teks klinis naratif berbahasa Indonesia, di mana satu dokumen dapat mengandung banyak entitas (N entitas), sehingga proses dekomposisi dilakukan secara batch untuk seluruh entitas yang terdeteksi oleh modul NER.



Selanjutnya QD (Langkah 4) akan diteruskan ke Langkah 5 dengan melakukan pemeriksaan query pada knowledge reservoir. Mekanisme Knowledge Reservoir diadopsi dari Gilani et al. (2025) untuk mengurangi biaya komputasi dan latensi. Jika sebuah query telah dipetakan sebelumnya dan tervalidasi oleh ahli, sistem akan **langsung mengambil hasil (cache hit)** tanpa melalui proses retrieval dan re-ranking ulang. Jika query belum pernah dipetakan (cache miss), sistem akan menjalankan pipeline berikutnya yaitu langkah 6 dan 7 (ensemble retrieval → filtering → re-ranking → validasi) dan menyimpan hasilnya untuk penggunaan mendatang.

### Konsep dan Implementasi Sparse Retrieval (Pencarian Jarang)

Metode ini berfokus pada **pencocokan kata kunci (keyword matching)** secara eksplisit, mirip dengan cara kerja mesin pencari tradisional.

- **Konsep Matematis:** Gilani menggunakan **SPLADE (Sparse Lexical and Expansion Model)** untuk sparse retrieval. SPLADE adalah model neural yang mempelajari representasi sparse.
  - Representasi ini berupa vektor dengan dimensi yang sama dengan kosakata (misal, ~30.000 untuk BERT), tetapi sebagian besar nilainya adalah **nol (sparse)**.

- Setiap dimensi yang bernilai tidak nol mewakili sebuah kata dari kosakata, dan nilainya menunjukkan bobot kepentingan kata tersebut untuk suatu dokumen atau query.
- Secara matematis, SPLADE menghitung bobot ( $w_j$ ) untuk setiap kata dalam kosakata ( $j$ ) menggunakan rumus aktivasi **log-saturation**:  $w_j = \log(1 + \text{ReLU}(z_j))$ . Di sini,  $z_j$  adalah logit (skor mentah) dari model untuk kata tersebut. Fungsi ReLU (Rectified Linear Unit) memastikan bobot tidak negatif, dan fungsi  $\log(1 + \dots)$  membantu menghasilkan representasi yang jarang (sparse).
- **Kelebihan:** Metode ini **interpretabel** (kita bisa tahu kata apa yang mempengaruhi hasil), **efisien** untuk pencarian skala besar (karena bisa di-index), dan sangat baik untuk mencocokkan istilah teknis atau singkatan secara eksak.
- **Kekurangan:** SPLADE tidak secara alami menangkap konsep yang sama tetapi diekspresikan dengan kata yang berbeda (**vocabulary mismatch problem**).
- **Implementasi oleh Gilani:** Model SPLADE menghasilkan representasi sparse untuk setiap konsep di knowledge base-nya. Saat ada query, representasi sparse query dibandingkan dengan representasi sparse konsep untuk menemukan kecocokan kata kunci terbaik.

### Konsep dan Implementasi Dense Retrieval (Pencarian Padat)

Metode ini berfokus pada **kemiripan makna (semantic similarity)**. Tujuannya adalah untuk menemukan konsep yang secara semantik mirip dengan query, meskipun menggunakan kata-kata yang berbeda.

- **Konsep Matematis:** Gilani menggunakan **SapBERT (Self-alignment Pretraining for BERT)** untuk dense retrieval. SapBERT adalah model yang mempelajari representasi vektor **padat (dense)** untuk entitas biomedis.
  - Vektor padat ini memiliki dimensi yang tetap (misal, 768) dan semua nilainya adalah angka real (bukan nol).
  - Tujuan dari SapBERT adalah untuk memetakan sinonim-sinonim dari sebuah konsep (misal, "heart attack" dan "myocardial infarction") ke titik yang **berdekatan** dalam ruang vektor.
  - Pelatihan SapBERT menggunakan **metric learning** dengan fungsi loss **contrastive**, seperti InfoNCE, yang memaksimalkan kesamaan antara representasi sinonim dan meminimalkannya dengan representasi konsep yang berbeda.

- Saat pencarian (inference), query dan setiap konsep diubah menjadi vektor padat. Kemiripan antara vektor query ( $e_q$ ) dan vektor dokumen ( $e_d$ ) dihitung menggunakan **cosine similarity**:  

$$S_{dense}(q,d) = (e_q \cdot e_d) / (||e_q|| ||e_d||)$$
Dokumen dengan skor cosine similarity tertinggi adalah hasil yang paling relevan secara semantik.
- **Kelebihan:** Metode ini unggul dalam menangkap **sinonim** dan konsep yang terkait secara semantik, mengatasi kelemahan utama sparse retrieval.
- **Kekurangan:** Kurang interpretabel, dan bisa "over-generalize", yaitu menganggap dua hal mirip padahal sebenarnya berbeda secara teknis (false positive).
- **Implementasi oleh Gilani:** Model SapBERT menghasilkan vektor padat untuk semua konsep di knowledge base. Saat ada query, vektor padat query-nya dibandingkan dengan semua vektor konsep menggunakan cosine similarity untuk menemukan kecocokan makna terbaik.

### 👉 Integrasi: Hybrid Retrieval ala Gilani

Gilani menggabungkan hasil dari kedua metode ini untuk mendapatkan hasil terbaik. Meskipun paper tidak menyebutkan metode fusi spesifik, pendekatan umum yang digunakan adalah **Weighted Reciprocal Rank Fusion (WRRF)**:

$$S_{final}(c) = (w_d / (K + r_d(c))) + (w_s / (K + r_s(c)))$$

- $S_{final}(c)$ : Skor akhir untuk sebuah kandidat konsep  $c$ .
- $r_d(c)$ : Peringkat (rank) konsep  $c$  dari hasil dense retrieval (1 = peringkat teratas).
- $r_s(c)$ : Peringkat (rank) konsep  $c$  dari hasil sparse retrieval.
- $w_d, w_s$ : Bobot yang diberikan untuk masing-masing metode (biasanya  $w_d + w_s = 1$ ).
- $K$ : Konstanta smoothing (biasanya 60).

Dengan metode ini, sebuah konsep yang mendapatkan peringkat tinggi dari **kedua** metode akan memiliki skor akhir yang sangat tinggi, sementara konsep yang hanya unggul di salah satu metode tetap memiliki kesempatan untuk muncul.

Singkatnya, pendekatan hybrid retrieval yang digunakan oleh Gilani secara cerdas menggabungkan presisi kata kunci dari sparse retrieval (SPLADE) dengan pemahaman semantik dari dense retrieval (SapBERT) untuk menghasilkan pemetaan konsep klinis yang lebih akurat dan komprehensif.

Sebagai penekanan, RAG yang diimplementasikan pada framework Gilani mencakup:

1. **Peningkatan Kualitas Input:** Proses **Query Decomposition** memastikan bahwa pertanyaan yang diajukan ke sistem retrieval sudah dalam format yang optimal.
2. **Pengambilan Pengetahuan Eksternal:** Komponen **Ensemble Retrieval** adalah inti dari proses *retrieval* yang mengambil informasi dari basis pengetahuan eksternal yang luas.
3. **Pembelajaran dan Optimasi:** Komponen **Knowledge Reservoir** memungkinkan sistem untuk "belajar" dari hasil yang telah divalidasi, sehingga menjadi lebih cepat dan efisien seiring waktu.

Dengan kata lain, **seluruh alur dari Query Decomposition hingga penyimpanan di Knowledge Reservoir adalah bagian dari strategi RAG yang diimplementasikan oleh Gilani** untuk memastikan pemetaan konsep klinis yang akurat, kontekstual, dan efisien

